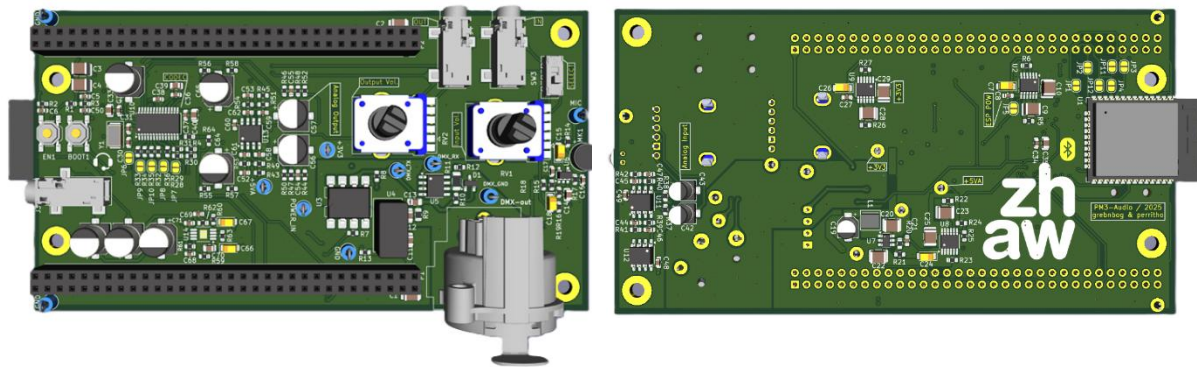




Hardware Report

Audio Processing and Ambient Light Control

Electronics Projects



Authors

Thomas Perri
Bogdans Grebnevs

Main supervisor

Patrick Rennhard

Secondary supervisor

André Huber

Date

21.11.2025

Abstract

This project presents the development of an audio-driven lighting control device built on an STM32 microcontroller and a CS4271 audio codec. The system accepts audio from a line-in connector, an on-board microphone, or an optional Bluetooth interface via an ESP32 module. The analog front end provides amplification, volume control, and filtering before the codec digitizes the signal at 48 kHz for processing. The STM32 extracts audio features to generate DMX512 lighting data, transmitted through an isolated RS-485 interface, while the audio stream is simultaneously converted back to analog and made available on line-out and headphone outputs. The hardware was implemented on a custom four-layer PCB and verified through a structured testing procedure covering power rails, audio paths, codec operation, and DMX output.

IND

Table of Contents

1.	Introduction	4
2.	Specifications, Functionality and System Overview	4
2.1.	Purpose of the System	4
2.2.	Basic Functions	5
2.3.	Optional Functions	5
2.4.	System Specifications	6
2.5.	User Interface and User Interaction	7
2.6.	Functional Block Diagram	8
3.	Development	9
3.1.	Implementation of a Dedicated 3.3 V Rail for the ESP32	9
3.2.	Analog input and filtering	11
3.3.	Codec circuitry and configuration	13
3.4.	Analog output and reconstruction filtering	14
3.5.	ESP32 Interconnect to STM32 and Codec	15
4.	Implementation	16
4.1.	Schematic Design	16
4.2.	PCB Layout	17
5.	Testing	18
6.	Project management	18

Appendix ?

1. Introduction

The objective of this project is to design and implement an integrated audio-processing and ambient-light-control device on a custom printed circuit board (PCB). The system is based on an STM32F429 microcontroller and a CS4271 audio codec. Its primary functions include digitizing an incoming audio signal, performing real-time processing to extract audio-level information and control ambient lighting, and then converting the processed signal back to analog for output through a line-out or headphone interface.

Beyond the core functionality specified by the course requirements, the project will explore additional features such as audio effects or enhanced signal-analysis algorithms (e.g., beat detection).

The development process comprises electronic circuit design, PCB layout, component assembly and soldering, firmware development, and comprehensive testing of both hardware and software. All tasks are planned to be completed within a single semester.

Certain hardware modules, reference designs, and firmware components were provided by the supervising professor to support and accelerate the development process.

2. Specifications, Functionality and System Overview

This section provides ^a general overview of the system, basic and optional functionalities as well as technical requirements and specifications.

2.1. Purpose of the System

The system can receive audio from either the on-board microphone or a conventional line-level input. After amplification and analog filtering, the signal is fed into the CS4271 codec, which converts it into a digital data stream. This digital audio is transferred to the STM32F429 microcontroller, where it is processed and used to drive the lighting control algorithms. In parallel, the audio path continues through the codec's DAC, generating an analog output that is made available on the line-out connector and the headphone jack.

2.2. Basic Functions

The core functionality defined by the course framework includes the following system capabilities:

- Receive audio signals from either a microphone input or a line-in source
- Apply an anti-aliasing filter to the input signal prior to A/D conversion
- Digitize the signal as 24-bit audio data using the CS4271 codec
- Transfer the digital audio stream to the STM32F429 microcontroller via the I2S interface
- Process the signal to determine the audio level and display the measurement results on the display
- Convert the processed signal back to analog form and apply a reconstruction filter
- Control the audio codec through an I²C connection
- Generating DMX512 frames at 250 kbit/s using the microcontroller's integrated UART

2.3. Optional Functions

Modern audio equipment is increasingly wireless, so relying only on analog audio jacks would limit the practicality of the device. To address this, an ESP32 module was added to the PCB to provide Bluetooth audio capability. The ESP32 acts as an alternative audio input/output interface without modifying the core system design.

Bluetooth Audio Functionality:

- Receive audio wirelessly via Bluetooth into the ESP32 module
- Decode the audio stream and forward it to the STM32 using an I²S connection
- Receive the processed audio signal from the STM32 and transmit it back over Bluetooth

This feature operates strictly as an additional audio path. It does not change the main STM32 audio-processing pipeline, and it does not affect the DMX or light-control functionality.

Additional optional features are planned, but they are purely firmware-based and do not require changes to the hardware. For this reason, they are not part of the hardware documentation.

2.4. System Specifications

▪ Electrical Specifications

- Analog supply voltage: 5 V
Used for the audio codec's analog parts and audio circuit components (e.g. op-amps).
- Digital supply voltage: 3.3 V
Required for the STM32 microcontroller, the digital section of the audio codec and the optional ESP32 module.

▪ Audio Subsystem

- Sampling rate: 48 kHz
Standard professional audio rate that balances quality and computational load.
- Audio resolution: 24 bit
Provides high dynamic range suitable for precise level detection and audio effects processing.
- Signal levels: Line-level audio for both input and output paths.

▪ DMX Communication Interface

- Bitrate: 250 kbit/s
Compliant with the DMX512 standard for stage and architectural lighting control.
- Electrical interface: RS-485 transceiver, differential bus.

▪ Mechanical Specifications

- PCB type: 4-layer PCB (course requirement)

▪ Interfaces and Connectivity

- Audio Interfaces: line-in and line-out / headphone
Analog input and output to the codec's ADC and from the codec's DAC respectively
- I²S interface for audio data transmission
- I²C interface for codec configuration
- UART interface for interaction between the microcontroller and the DMX output
- Lighting Control: DMX output

▪ Additional Requirements for Optional ESP32 Bluetooth Feature

- Additional 3.3V power supply for the ESP32 microcontroller
- Additional routing for I²S lines
- Additional UART connection as a backup ESP32 – STM32 communication channel

2.5. User Interface and User Interaction

This section describes the interfaces through which the user interacts with the device.

Audio Input Options

The device supports several ways of providing an audio signal:

- Line-in (standard analog input)
- On-board microphone integrated on the PCB
- Bluetooth audio input through the optional ESP32 module

The user can select between the analog audio sources (line-in or microphone) using a DIP switch on the PCB. Bluetooth input does not require hardware switching, as it is handled entirely in software.

Audio Output Options

The processed audio signal can be sent to:

- Line-out (analog analog input)
- Headphone jack
- Bluetooth audio output via the ESP32 module

Analog Volume Control

Two potentiometers are placed on the PCB for adjusting:

- The input gain/volume of the analog line-in or microphone signal
- The headphones output level

These controls operate only in the analog part of the audio path and therefore do not affect Bluetooth audio, which remains fully digital. In practice, this is not a limitation because most Bluetooth audio sources and devices (e.g., smartphones, laptops, headphones) include their own built-in volume control.

Analog Volume Control

All remaining interactions—such as managing Bluetooth connections, adjusting light-control settings, and accessing additional features—are handled through a touch display. This interface provides flexibility and can be fully defined during firmware development. It is designed to cover all necessary user interactions, so no further hardware controls are required.

2.6. Functional Block Diagram

The functional block diagram illustrates the complete signal flow within the hardware system. It shows the path from the audio input, through the microcontroller and audio codec, to the analog audio output and the DMX lighting output.

The system supports multiple audio input options. In the analog case, the signal enters either through the on-board microphone or the line-in connector. The microphone signal is first amplified by a preamplifier to reach the level required by the codec, while the line-in signal bypasses this stage. Both signals then pass through the input-selection block and the input volume control.

Next, the selected analog signal is routed to an anti-aliasing filter. This second-order low-pass filter removes high-frequency components above the useful audio range. By doing so, it prevents aliasing artifacts when the codec's ADC samples the signal.

After filtering, the signal is converted into a digital data stream by the codec's ADC. The codec sends the digitized audio to the STM32 microcontroller via the I²S interface. The microcontroller processes the audio samples – for example, to extract level information or to apply optional audio effects. The resulting audio data is then returned to the codec, where the DAC converts it back into an analog signal.

The analog output passes through a reconstruction filter that suppresses high-frequency copies of the original signal frequencies produced by the DAC. The signal is then routed to the line-out connector. In parallel, it also goes through an output volume control stage and a headphone power amplifier before reaching the headphone jack.

At the same time, the microcontroller generates DMX512 frames for lighting control based on the processed audio data. These frames are transmitted via a UART interface to the DMX driver, which both galvanically isolates the DMX interface from the rest of the system and converts the UART signal into a differential RS-485 signal. The resulting isolated signal is available at the DMX output connector and can be used to drive external lighting equipment.

In addition to the standard analog path, the system provides an optional fully digital Bluetooth audio path. In this case, the ESP32 module receives audio wirelessly, decodes it, and forwards the digital audio stream to the STM32 over I²S. After processing, the STM32 returns the audio to the codec for conversion and output, or back to the ESP32 for wireless transmission. It is also possible to use Bluetooth only as an input or only as an output, combining the analog and digital paths as needed.

A second communication option between the ESP32 and STM32 was added as a design-time precaution. During PCB design, it was uncertain whether the primary ESP32-to-STM32 I²S link would work reliably, and the project timeline did not allow for a redesign if problems appeared. Therefore, an alternative scheme was included: the ESP32 can interface directly with the

codec over ^{I2S}, while a separate UART link handles communication with the STM32. Only one of these schemes is used in practice.

3. Development

For this project, the most critical circuit blocks were pre-designed and reviewed by the instructors. These included the regulated 5 V, 5 VA-Analog (5 VA), and 3.3 V power rails, the DMX interface circuitry, the main external connectors, and the mandatory components such as the audio codec, the headphone amplifier, and the microphone front end. The initial design used a single linear regulator for the 3.3 V rail, but after system analysis we added a second regulator to support the additional current demands introduced by the ESP32.

The remaining circuits required additional development work, including:

- Implementation of a dedicated 3.3 V rail for the ESP32
- Designing analog input stage and anti-aliasing filter
- Codec configuration and support circuitry
- Designing analog output stage and reconstruction filter
- Interfacing between the ESP32, STM32, and audio codec

3.1. Implementation of a Dedicated 3.3 V Rail for the ESP32

The instructors provided fully designed power rails for 5 V, 5 VA, and 3.3 V. However, during the system planning process it became clear that the existing 3.3 V rail – based on a linear regulator – was not suitable for powering the ESP32 together with the rest of the digital circuitry. The ESP32 can draw up to approximately 500 mA under full load, especially during wireless activity, which would push the original regulator close to or beyond its safe operating limits. In addition, since the entire system is powered from USB (5 V at roughly 500 mA), the total available power is limited to about 2.5 W, making efficient power distribution essential. Under these constraints, relying on a single 3.3 V regulator was not feasible.

To address this, we added a second 3.3 V regulator dedicated exclusively to the ESP32 (see Figure below). Splitting the load across two regulators provides sufficient current headroom, avoids overload, and reduces thermal stress. It also improves stability for the ESP32, whose wireless transmissions can cause rapid and significant current spikes.

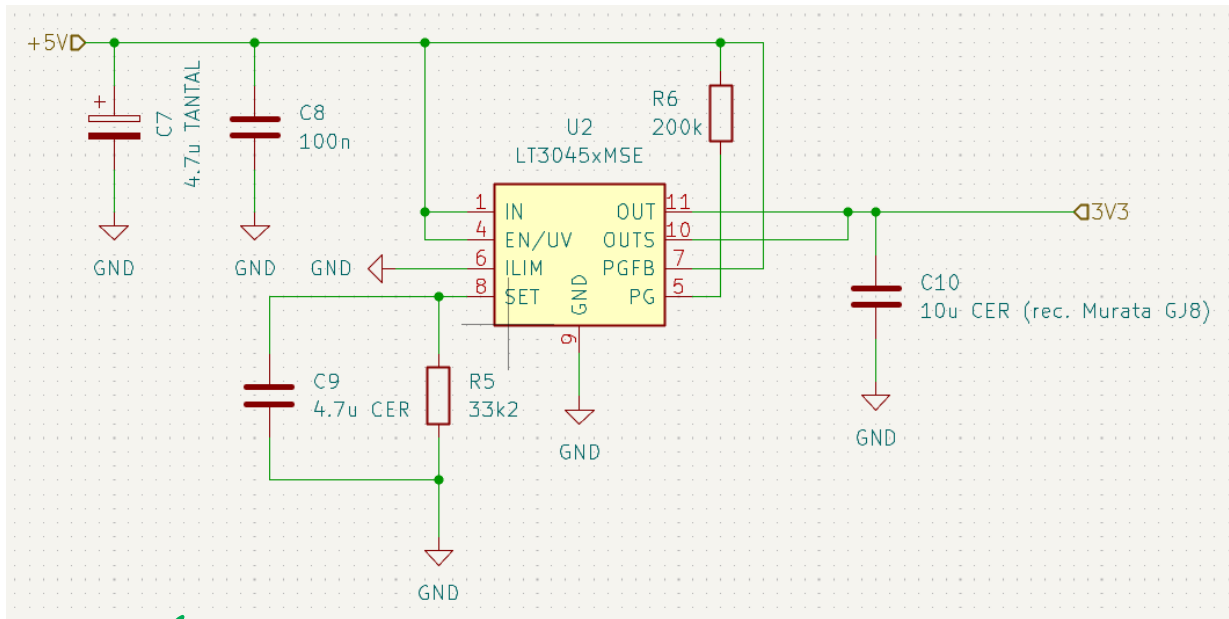


fig. X: . . .

To further smooth these current spikes, we added substantial decoupling capacitance to the dedicated ESP32 rail (see Figure below). Capacitors were placed both at the regulator output and close to the ESP32 power pins. This local energy buffering reduces high-frequency fluctuations, stabilizes the supply during wireless bursts, and increases the overall reliability of the power distribution network.

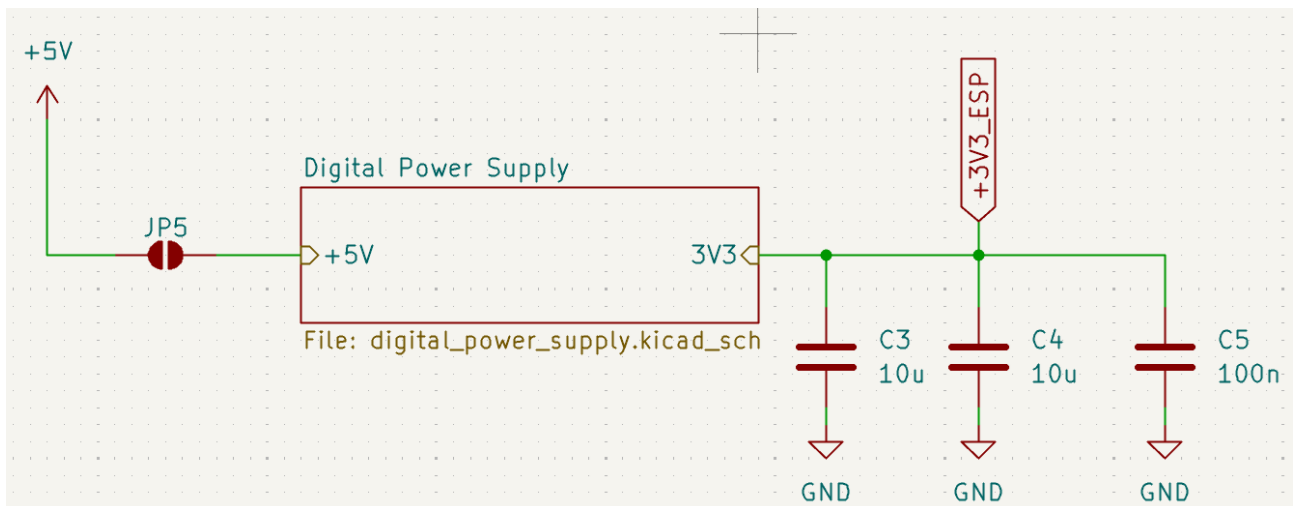
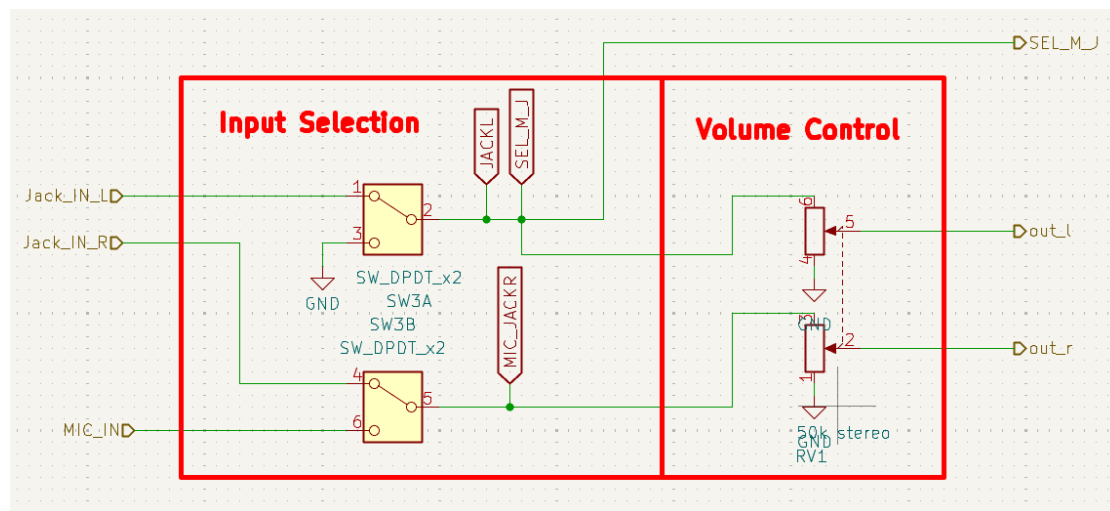


fig. X: . . .

3.2. Analog input and filtering

The audio input stage (see Figure below) supports two analog sources: the on-board microphone and a 3.5 mm audio jack. A selection switch allows either the microphone signal or the jack signal to be routed to the volume control stage. The switch position is monitored by the STM32 through the SEL_M_J signal, enabling the firmware to determine which source is currently active.

The input volume control is implemented using a single 50 k Ω stereo potentiometer. Using a stereo component ensures that both audio channels maintain identical gain settings, keeping the input signal levels balanced.



After the volume control, the signal is passed to the analog input filter in front of the codec (see Figure below). This stage does more than simple filtering – It also prepares and conditions the signal so that it matches the input requirements of the codec. First, the DC component is removed using coupling capacitors. The signal is then shifted and centered around the codec's VCOM reference voltage using an op-amp buffer. The VCOM reference itself is generated internally by the codec.

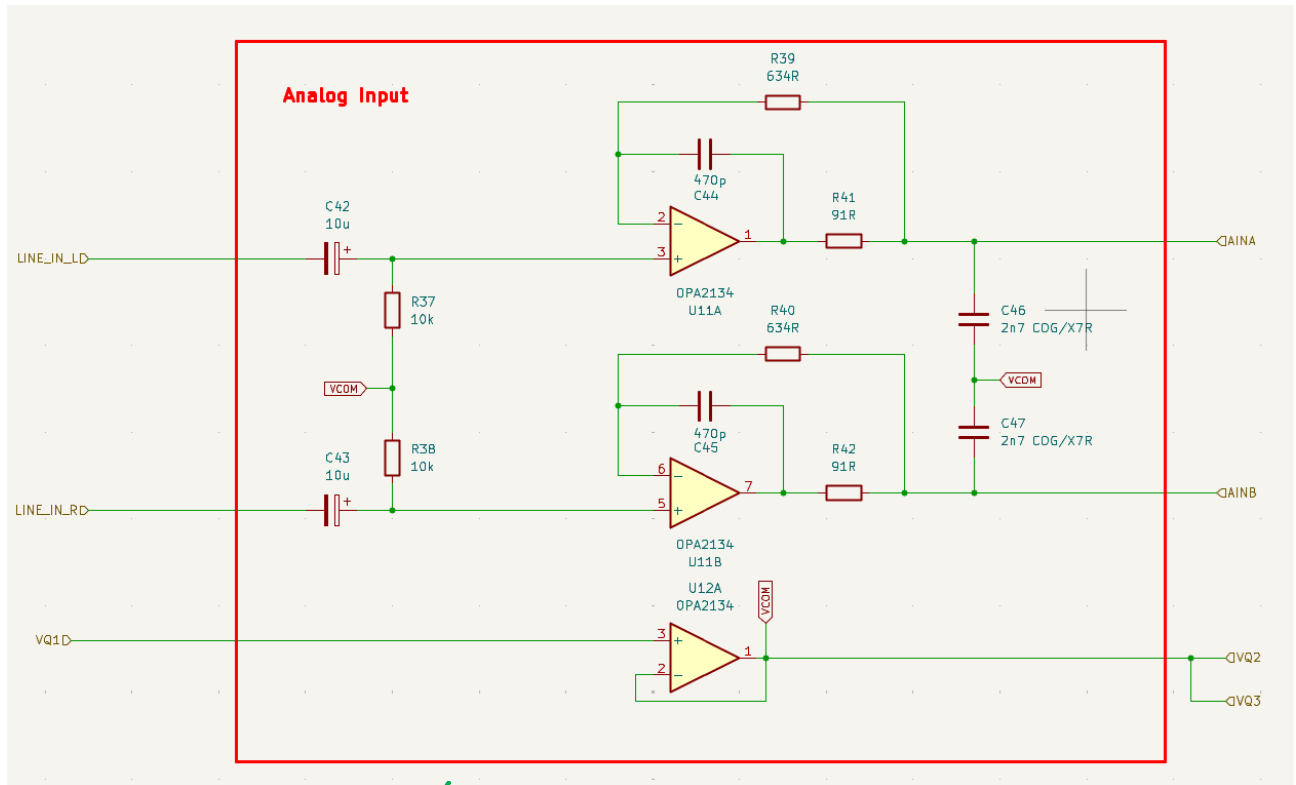
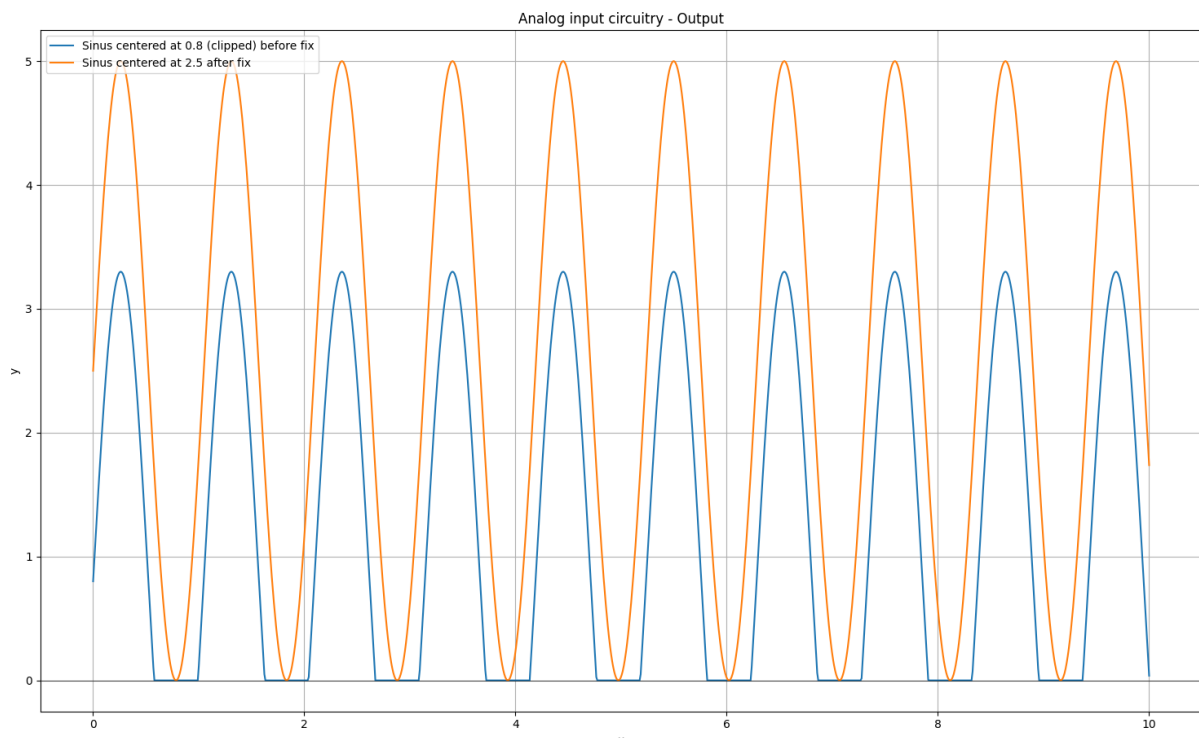


Fig. X. ...

The analog front end was designed based on the instructors' guidelines and the codec datasheet, and subsequently simulated in LTspice to verify correct behavior (see Appendix).

Unfortunately an error was introduced during schematic design: the signal nets before and after the VCOM buffer amplifier were given the same name. As a result, the buffer output was unwittingly tied directly to the codec's VCOM pin, causing the amplifier to load the reference supply instead of buffering it. This issue became visible during measurements, where the filter output showed clipping on the lower part of the waveform, indicating that the VCOM reference was not at its intended 2.5 V level (see Figure below).

The issue was resolved by cutting the incorrect PCB traces and reconnecting the affected signals using enameled bodge wires. After this correction, the analog filter and input stage operate as intended (see Figure below). Further details related to the correction and measurement results are documented in the Hardware Test Report in the appendix.



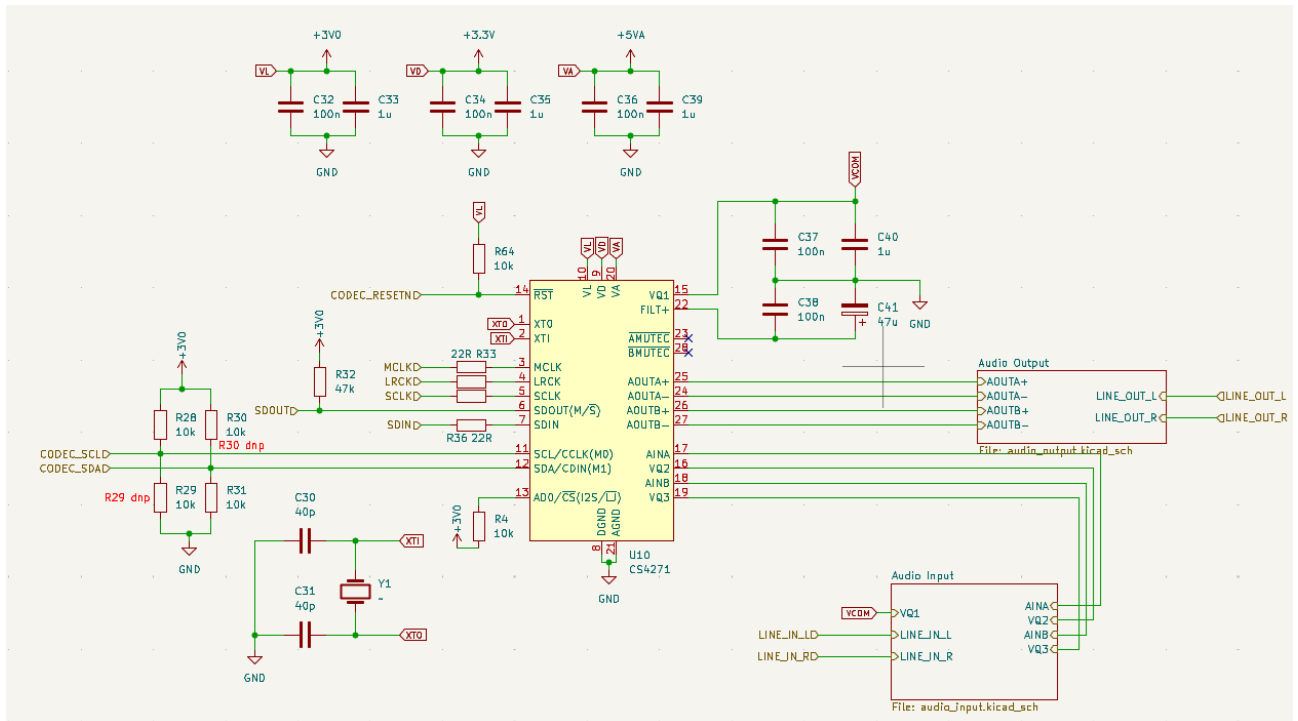
3.3. Codec circuitry and configuration

The codec circuitry itself was relatively straightforward, as most of the required components and connections follow directly from the manufacturer's reference design. The schematic was therefore largely adapted from the codec datasheet, with only a few configuration decisions left to be made during development.

The main configuration-related elements were the M0 and M1 mode-select pins. Their logic levels are defined using pull-up or pull-down resistors. Since we did not want to commit to a single fixed configuration during the design phase, we added footprints for both pull-up and pull-down resistors (see Figure below). This gives flexibility to choose the final settings during bring-up or testing.

For normal operation, we selected the configuration M0 = '1' and M1 = '0'. This enables single-speed mode with a sampling range of 4–50 kHz and disables the de-emphasis filter. De-emphasis is used only when a corresponding pre-emphasis filter has been applied earlier in the signal chain in order to compensate for boosted high-frequency components. Since our system does not use pre-emphasis, enabling de-emphasis would be unnecessary.

The codec's RESET pin is connected to the STM32's 3.3 V rail through a 10 kΩ pull-up resistor. In addition, the pin is routed to a microcontroller GPIO. (see Figure below) This allows the STM32 to trigger a software-controlled reset of the codec if required, for example in case of a communication fault or configuration error.



3.4. Aanalogue output and reconstruction filtering

The codec provides a differential analog output, which must be converted back into a single-ended signal before it can be routed to the line-out and headphone amplifier. In addition, the signal requires reconstruction filtering. Because the codec has finite resolution in both amplitude and time, its DAC output consists of small discrete steps rather than a perfectly smooth waveform. As a result, the raw output contains high-frequency components and sharp edges that are not desirable in an audio signal. (see Figure below)

The reconstruction filter (see Figure below) smooths these edges and removes out-of-band noise generated by the DAC's sampling process. The filter ensures a clean, continuous output waveform that stays within the audible frequency range and meets the requirements for both line-level output and headphone amplification.

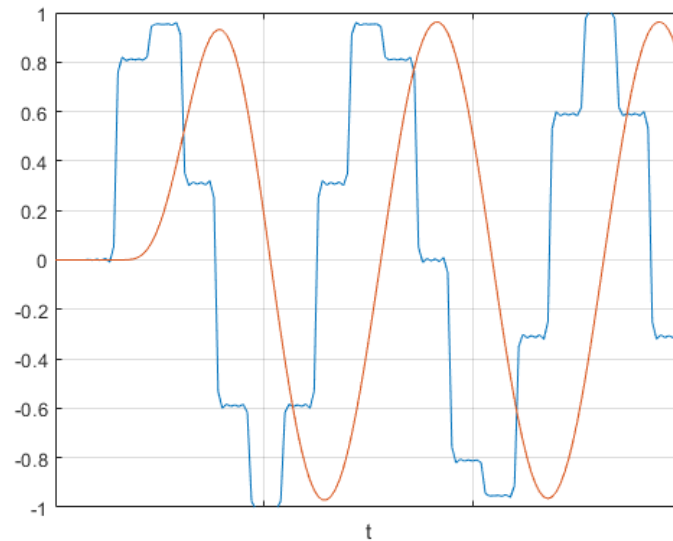


Fig. X

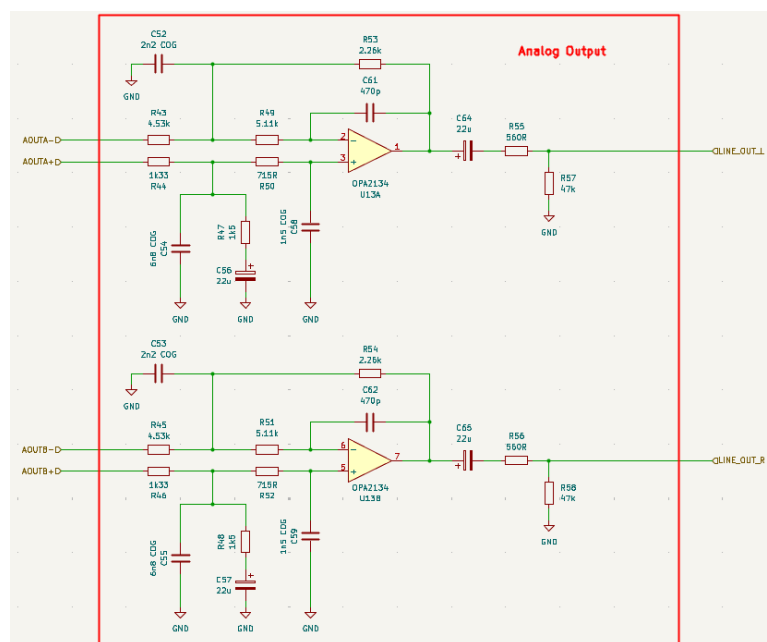


Fig. X

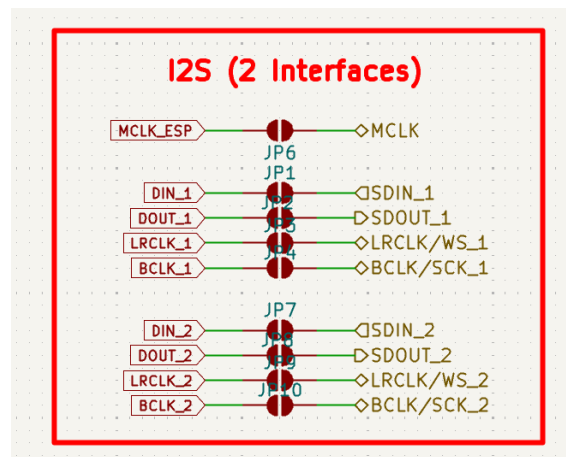
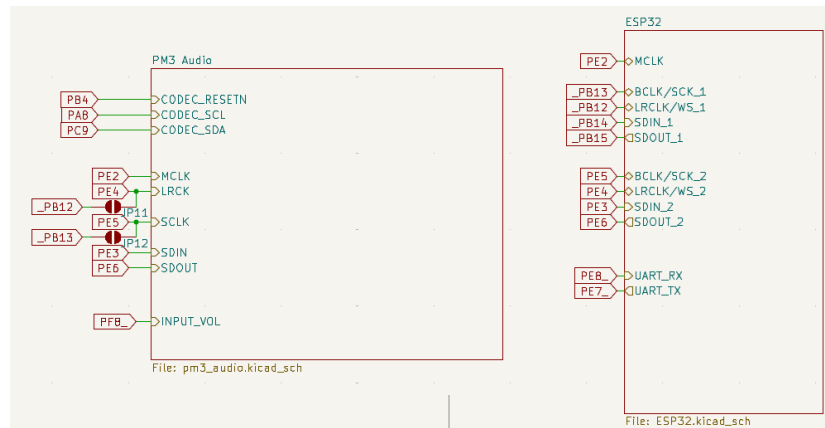
3.5. ESP32 Interconnect to STM32 and Codec

The main additional feature of the board is the Bluetooth audio capability provided by the ESP32. The ESP32 can function either as a Bluetooth audio source—receiving audio wirelessly and forwarding it as I²S data—or as a Bluetooth audio sink, transmitting audio that has been processed by the STM32.

Establishing the correct interconnections between the ESP32, STM32, and the codec is not trivial. Because I²S signals are high-speed and sensitive to loading, only one device may drive the codec's digital audio input at a time. To avoid interference or bus contention, the possible configurations are selected through solder-jumper pads, which allow the desired signal route to be fixed reliably during assembly. (see Figures below)

The jumper configuration supports three distinct operating schemes:

- STM32 → Codec: The STM32 is the sole audio source driving the codec via a direct I²S connection (Standard mode)
- ESP32 → Codec: The ESP32 sends I²S audio directly to the codec, bypassing the STM32. (Backup configuration or direct Bluetooth-to-codec mode)
- ESP32 ↔ STM32 → Codec: The ESP32 exchanges audio with the STM32 over I²S, and the STM32 then forwards the processed audio to the codec. (Primary Bluetooth processing mode)



4. Implementation

This section describes the implementation stage of the project, from schematic creation to PCB layout and final design checks.

4.1. Schematic Design

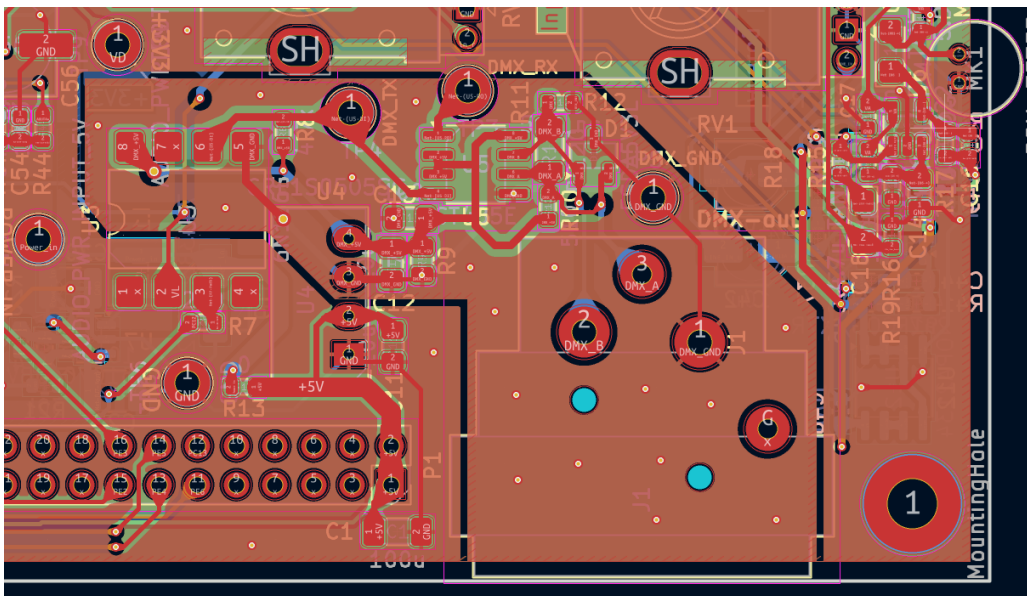
The design process began by grouping all components into clearly defined functional blocks. For each block, small local circuit layouts were drafted to maintain an organized overview and to reduce the risk of major PCB restructuring later on. Several parts of the system – such as

portions of the power rails and the DMX interface circuitry – were provided by the instructors. These sections were adopted without modification to ensure compatibility with the rest of the course material.

4.2. PCB Layout

After completing the schematics, the PCB layout was created in KiCad. (see Layout in appendix) Component placement followed the overall signal flow: audio-related components were positioned close to the codec to minimize trace length and noise pickup, while the DMX output stage was placed near its connector to keep the isolated RS-485 traces short and robust.

To avoid interference between subsystems, the analog and digital regions of the board were separated as much as possible. Their ground planes were connected at a single defined point to prevent unwanted ground loops. The isolated DMX ground was also routed carefully, making sure that copper zones remained continuous and were not unintentionally split by traces.



Standard layout practices were applied throughout the design. Decoupling capacitors were placed close to the supply pins of each IC, wide traces were used for power distribution, and sharp trace corners were avoided to maintain signal integrity. Thermal reliefs were added to ground pads to make soldering during assembly easier.

Before sending the design for manufacturing, KiCad's design rule checks were run to verify clearances, track widths, and net continuity. The 3D viewer was then used to confirm mechanical fit, component spacing, and overall layout consistency. Finally, both the schematic and PCB layout were reviewed and approved by the instructors.

5. Testing

After completing the design and assembly of the device, a series of tests was carried out to verify correct functionality of the hardware. A detailed test protocol, including the full test procedure, measurement results, and evaluation tables, is provided in the appendix.

where?

6. Project management

For the time planning a Gant chart has been used (see Figure below and the appendix).

....++

AUDIO BOARD (HS25)

Thomas Perri & Bogdans Grebnevs

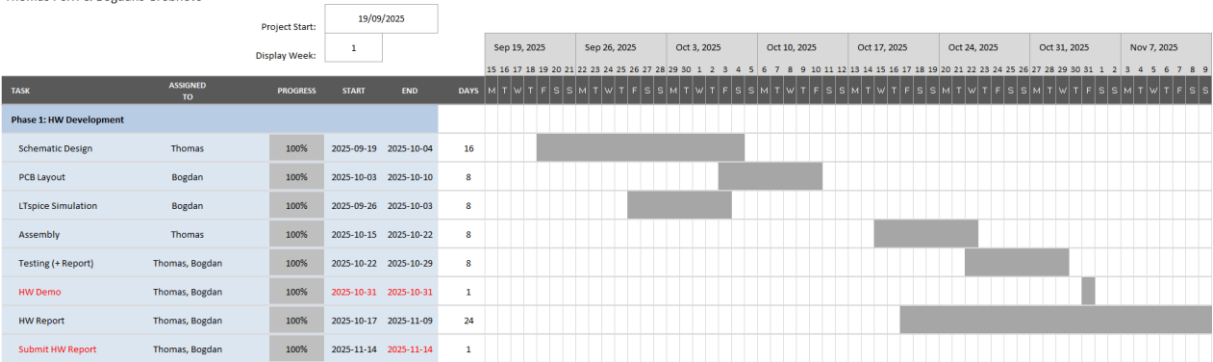


fig. X.